

PAPER

Network Intrusion Detection Method Based on Semi-Supervised Learning and Random Forest

Junji LI^{†a)}, *Member*, Haohang SUN^{†b)}, Hui DU^{††c)}, Lin LI^{†d)}, and Zelin ZHANG^{†††e)}, *Nonmembers*

SUMMARY Network Intrusion Detection Systems (NIDS), as critical tools for defending against malicious attacks, still have several limitations, such as false positives and false negatives, limited adaptability to new attacks, and scalability issues. To enhance the adaptability and real-time detection capabilities of NIDS for novel attacks, this paper proposes a network intrusion detection method based on semi-supervised learning and random forest. Firstly, an autoencoder is used to reduce the dimensionality of the NSL-KDD dataset, increasing robustness against high-dimensional complex data. Secondly, a semi-supervised learning framework is established through the DBSCAN clustering method, improving the model's performance and stability. Finally, multi-layer perceptron (MLP) is employed to select data features and extract an optimal feature subset, and random forest (RF) classifier is constructed to reduce computational complexity and effectively boost model performance. Experimental results show that the proposed method improves accuracy on the NSL-KDD and UNSW-NB15 datasets by 8.92% and 6.1%, respectively, which demonstrates strong generalization ability and robustness, achieving precise and efficient detection and classification of network intrusions and attack types.

key words: network intrusion detection, semi-supervised learning, random forest, multi-layer perceptron, autoencoder

1. Introduction

Network intrusion detection technology is essential for cybersecurity, enabling real-time monitoring and identification of potential threats to prevent attacks such as data breaches and system damage. However, with the increasing complexity and diversity of network attack methods, the growing pressure of real-time processing for massive data flows, and the continuous escalation of adversarial attacks, intrusion detection technology faces severe challenges. When dealing with minority class samples and new attack types, most models tend to perform poorly, which is also one of the pressing issues to be addressed.

As one of the key methods for safeguarding network security, network intrusion detection technology traditionally encompasses signature-based detection, anomaly detection, rule-based detection, and statistical detection. Traditional

methods perform well in handling known attacks but are inadequate in detecting complex, variant attacks and processing large-scale data, especially in addressing unknown threats and reducing false positive rates, revealing notable limitations.

With ongoing research and applications of machine learning in data mining and network intrusion detection, detection performance has seen significant improvement. LSTM, L-SVM, Q-SVM, and other approaches have been applied for detection, yet issues such as insufficient robustness and inadequate feature extraction remain. While existing machine learning methods can effectively learn latent data features and automate detection, the ever-growing data volume and the emergence of novel attacks limit the applicability of network intrusion detection in complex system environments.

Therefore, this paper proposes a network intrusion detection method based on semi-supervised learning and random forest. In a semi-supervised learning environment, an autoencoder is used to reduce data dimensionality, a multi-layer perceptron (MLP) is employed to obtain the optimal feature subset, and a random forest classifier is used for classification and detection. The specific dynamic collaboration workflow and innovative mechanisms are as follows: First, by training an autoencoder, high-dimensional data is effectively mapped to a lower-dimensional space, learning and representing latent features to reduce data complexity and address the limitations of feature extraction with large-scale, complex data. Second, DBSCAN clustering is employed, using a small set of labeled data to generate soft labels for the majority of unlabeled data, thereby mitigating the drawbacks of supervised learning. The model is trained within a semi-supervised learning framework to overcome the inherent limitations of fully supervised approaches. Finally, a multilayer perceptron (MLP) neural network is employed to filter data features, extracting an optimal feature subset to improve the classification performance of the random forest classifier. This enhances its generalization ability when dealing with minority class samples and new attack types. Experimental results demonstrate that this method achieves strong intrusion detection performance, offering significant practical value.

2. Related Work

In the field of Network Intrusion Detection Systems (NIDS), traditional methods initially held a dominant position. Fa-

Manuscript received December 16, 2024.

Manuscript revised February 25, 2025.

Manuscript publicized May 2, 2025.

[†]Taiyuan University of Science and Technology, China.

^{††}Hangzhou Shinevai Technology Co., Ltd., China.

^{†††}Peking University Yangtze River Delta Institute of Optoelectronics, China.

a) E-mail: lijunji@tyust.edu.cn

b) E-mail: 963095352@qq.com (Corresponding author)

c) E-mail: 13811634201@139.com

d) E-mail: 2023077@tyust.edu.cn

e) E-mail: zhangzl_fdu@163.com

DOI: 10.23919/transcom.2024EBP3204

rooq Anjum et al. [1] proposed applying signature detection technology to intrusion detection in wireless ad hoc networks, which can effectively detect known signature attacks with low false positive rates. However, it cannot detect unknown signature attacks and is greatly affected by routing protocols. The detection performance in wireless ad hoc networks is limited and constrained by node resources. Bart Haagdoorens et al. [2] proposed applying multithreading techniques to signature-based network intrusion detection, enabling efficient use of multiprocessor hardware resources to improve detection performance, with significant gains on dual-CPU platforms. However, certain designs incur high overhead and provide limited performance improvements, constrained by the number of concurrent threads on the test platform and the granularity of state information protection. Vinod Kumar et al. [3] used SNORT to develop a signature-based intrusion detection system capable of effectively detecting known attacks, offering high customizability and flexibility. However, its ability to detect unknown attacks is limited, and its performance is affected by network traffic and other factors. R. Sekar et al. [4] combined specification-based and anomaly-based detection into a novel approach, effectively detecting probe and denial-of-service attacks with low false-positive rates. Nevertheless, this approach suffers from high false-positive rates and struggles to adapt effectively to novel attacks.

With the application and research of machine learning in network intrusion detection, efforts aim to enhance detection accuracy and real-time responsiveness. Many researchers have focused on optimizing networks. Fengqin Zuo et al. [5] utilized an improved Osprey Optimization Algorithm to select the most relevant and important features in network traffic, reducing dimensionality; however, the model faces robustness issues against complex adversarial attacks. Jay Sinha et al. [6] proposed a deep learning model based on convolutional neural networks and bidirectional long short-term memory networks to capture both spatial and temporal features in data, but its detection performance for minority attack classes needs to be improved. Jiawei Du et al. [7] developed an NIDS model, CNNLSTM, combining convolutional and long short-term memory networks to extract spatiotemporal features from network traffic for intrusion detection. However, this model has limited generalization capacity when faced with novel attack types or complex network environments. Osama Faker et al. [8] integrated big data with deep learning, employing a deep feedforward neural network (DNN), random forest (RF), and gradient boosting tree (GBT) to classify data by extracting features most relevant to intrusion detection. However, the feature selection approach used has limitations, leaving room for further improvement in model performance. Yukun Wu et al. [9] introduced an intrusion detection system combining a deep belief network (DBN) with a feature-weighted support vector machine (WSVM) to extract deep features and reduce dimensionality, though its classification performance is impacted by data imbalance. Ammar Odeh et al. [10] introduced an integrated deep learning architecture that enables hierarchical

computation and learning. However, the model's performance may overly depend on the feature distribution and data quality of a single dataset, and its generalization ability needs to be improved. Optimizing networks can significantly improve convergence speed, reduce overfitting, and enhance inference efficiency; however, such optimizations cannot resolve issues with data quality, such as noise, label errors, or model bias from sample imbalance.

In addition, many researchers have made advancements in analyzing data features for intrusion detection. Saeid Soheily-Khah et al. [11] proposed a hybrid data-mining-based intrusion detection method that combines random forest and k-means clustering for data feature processing and analysis, though further improvements are needed to address complex, novel attacks. Manjula C. Belavagi et al. [12] conducted numerical transformations on categorical data to evaluate different supervised machine learning methods, finding that the random forest classifier outperformed others. However, as their experiments relied on a single dataset, the model may be overly dependent on that dataset's feature distribution and data quality, potentially limiting robustness. Janarthanan et al. [13] employed various machine learning models and feature sets to study feature importance, enhancing intrusion detection performance. However, limitations in the feature selection and extraction process resulted in certain key features not being effectively utilized. Kumar et al. [14] proposed an ensemble-based IDS that demonstrated higher accuracy on the UNSW-NB15 dataset than traditional models. However, feature selection may not have fully accounted for feature interrelationships, resulting in some valuable features being overlooked. Zhiqiang Zhang et al. [15] proposed a Multiple Integration Model (MIM) for feature selection and attack classification. MIM employs a Rule and Priority-based Integration Strategy (RPES), enhancing the stability and reliability of the integrated model. However, it requires substantial computational resources, limiting its applicability, and its effectiveness and suitability for mildly imbalanced or balanced samples remain to be verified. Abiodun Ayantayo et al. [16] proposed a novel deep learning architecture using a deep fusion mechanism aimed at encouraging deep learning models to capture relationships among features. However, this approach may encounter the "closed-world" assumption issue when facing novel attacks. Sydney M. Kasongo et al. [17] conducted studies using various machine learning (ML) methods on the UNSW-NB15 dataset, examining the impact of the feature selection phase. However, the classifier's performance was suboptimal for minority class samples. Cosimo Ieracitano et al. [18] developed an IDS driven by statistical analysis and autoencoder (AE) technology to extract highly optimized and strongly correlated features for classification, but the model's overall generalizability is impacted by dataset imbalance. Vinayakumar Ravi et al. [19] proposed the Scale-Hybrid-IDS-AlertNet (SHIA) framework, with extensive dataset analysis and processing, but its ability to rapidly adapt to new attack patterns and features remains limited, requiring continuous model updates to maintain detection capabilities for novel attacks. Feature

selection and dimensionality reduction techniques can help extract essential information, improving model accuracy and efficiency in feature analysis. However, feature analysis cannot capture all potential attack patterns, especially unknown or mutated attacks, meaning that improvements in feature engineering cannot fully replace the need to ensure data quality.

In summary, current network intrusion detection systems face limitations in feature extraction. Effective features aid in identifying and extracting key information relevant to potential attacks, thereby enhancing detection accuracy, reducing false positives, and improving model interpretability. Additionally, traditional detection methods rely on supervised learning, where constructing a large, fully labeled dataset is challenging, error-prone, and time-consuming. Once data is unlabeled, the learning model cannot leverage its potential information. Given these challenges, this paper proposes a network intrusion detection method based on semi-supervised learning and random forest. Semi-supervised learning is employed to effectively extract useful information from both labeled and unlabeled data. A multilayer perceptron (MLP) is used to enhance the performance of the random forest classifier in network intrusion detection, aiming to overcome the limitations of existing models in feature extraction and supervised learning, while improving performance on minority and novel samples.

3. Algorithm Description

The overall design and training process for the network intrusion detection model, based on semi-supervised learning and random forest, is illustrated in Fig. 1.

This study primarily includes four key components: an autoencoder, semi-supervised learning, a multilayer perceptron, and a random forest classifier. First, the autoencoder maps high-dimensional data to a lower-dimensional space, helping to identify and extract key information related to potential attacks, thereby enhancing detection accuracy, reducing false positives, and making the model more interpretable. Second, clustering methods enable semi-supervised learning, effectively addressing the limitations of traditional supervised learning. Finally, the MLP is used to obtain an optimal feature subset to train the random forest classifier, enhancing its ability to detect minority and novel attack types. The integration of these four components provides a solid foundation for constructing a more effective network intrusion detection system.

3.1 Autoencoder Module

The autoencoder, as an unsupervised learning model, was initially proposed by Hinton and Salakhutdinov [20]. It aims to learn latent low-dimensional feature representations from input data and extract key information through a compression and reconstruction process. This process not only helps with effective feature extraction but also lays a more solid foundation for subsequent model training, thereby enhancing

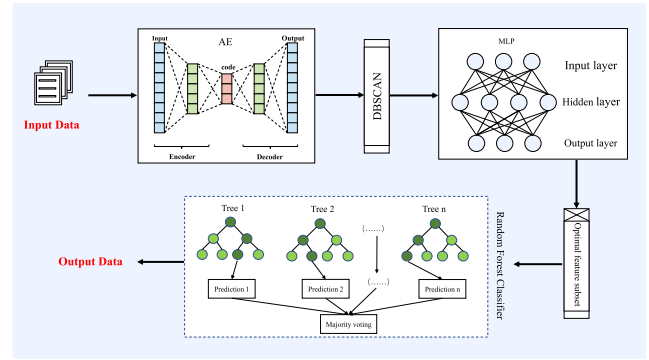


Fig. 1 Overall flowchart of the model.

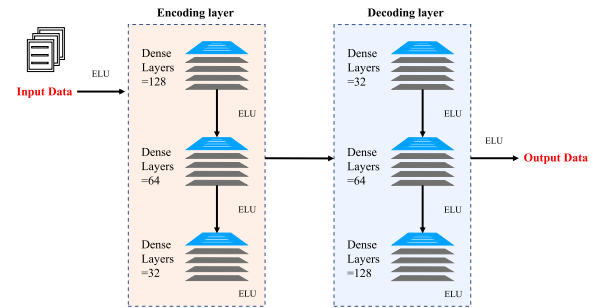


Fig. 2 Autoencoder module.

the model's performance and generalization ability. The autoencoder used in this paper is shown in Fig. 2.

Before training the autoencoder, the dataset undergoes preprocessing to ensure data quality and suitability. The data preprocessing process includes several steps: first, loading the dataset and removing the irrelevant "difficulty_level" column to reduce the interference of noisy data on model training. Then, one-hot encoding is applied to the categorical features in the dataset, converting them into multiple binary features in numerical form so that the model can process them; then, encoding the label column into numeric format, assigning a unique integer value to each class; and finally, using the StandardScaler to standardize the features, adjusting them to have a mean of 0 and a variance of 1, which improves the efficiency and stability of model training. The standardization formula is as follows:

$$X' = \frac{X - \mu}{\sigma} \quad (1)$$

Where: X is the original feature value, μ is the mean of the feature, σ is the standard deviation of the feature, and X' is the normalized feature value. This step aims to convert each feature into a standard normal distribution with a mean of 0 and a variance of 1, helping to reduce the impact of differences in feature scales on model training.

In addition, when addressing the data imbalance issue, the SMOTE oversampling technique is used to generate synthetic samples for the minority class, ensuring that the number of minority class samples is increased to 80% of the majority class. Moreover, random undersampling is ap-

plied to balance the sample size of the majority class, further mitigating the impact of class imbalance on the model's performance.

Next, the preprocessed dataset is input into the autoencoder for training. The autoencoder used in this paper is shown in Fig. 2. As an unsupervised learning model, the autoencoder is designed to learn potential low-dimensional feature representations from the input data and to extract key information through a compression and reconstruction process. This process not only aids in effective feature extraction but also establishes a solid foundation for subsequent model training, thus enhancing the model's performance and generalization ability. The autoencoder consists of two main components: the decoder and the encoder, explained in detail as follows:

Encoder: Maps input x to a latent space representation h through a function $h = f(x)$, where the dimension of h is typically smaller. This process can be implemented using the hidden layers of a neural network. The encoder used in this paper consists of three fully connected neural network layers, containing 128, 64, and 32 neurons, with the activation function being ELU. The formula is as follows:

$$h = f(x) = \sigma(Wx + b) \quad (2)$$

Where: x is the input data, W and b are the weights and biases of the encoder, σ is the activation function (such as ReLU or Sigmoid), h is the latent space representation.

Decoder: Generates the reconstructed input $\hat{x}$ from the latent representation h through a function $\hat{x} = g(h)$, implemented by a decoder neural network. The decoder used in this paper is symmetrically structured with 32, 64, and 128 neurons, and the activation function is also ELU. The formula is as follows:

$$\hat{x} = g(h) = \sigma'(W'h + b') \quad (3)$$

Where: h is the latent representation output by the encoder, W' and b' are the weights and biases of the decoder, σ' is the activation function (such as Sigmoid or linear activation), $\hat{x}$ is the reconstructed output.

Loss function: The objective of the autoencoder is to minimize the difference between input x and reconstructed output $\hat{x}$, typically using Mean Squared Error (MSE) as the loss function:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|^2 \quad (4)$$

Where: N is the number of training samples, x_i and $\hat{x}_i$ are the input and reconstructed output of the i -th sample, respectively.

To enhance the generalization ability of the autoencoder, this paper introduces a sparsity regularization term to prevent model overfitting. The formula for the sparsity regularization term is as follows:

$$\Omega_{sparsity} = \sum_{r=1}^J KL\left(\frac{A_r}{A_i^r}\right) \quad (5)$$

Where A_i^r represents the average activation of the i -th neuron, and A_r represents the desired activation value. By introducing the sparsity regularization term, the autoencoder is able to learn a more sparse feature representation, thus improving the model's generalization performance.

In the proposed network intrusion detection model, the autoencoder serves as the core component for feature extraction, and its output latent representation z is passed to the subsequent DBSCAN clustering model and multilayer perceptron (MLP). This integration not only reduces the dimensionality of the data but also enhances the training efficiency and detection performance of the downstream models.

3.2 Semi-Supervised Learning Module

Semi-supervised learning combines the strengths of supervised and unsupervised learning by utilizing a small amount of labeled data alongside a large amount of unlabeled data, reducing labeling costs. At the same time, it enhances the model's generalization ability and robustness, mitigating the risk of overfitting, especially in scenarios with scarce labeled data. By introducing additional information into the learning process, it improves overall performance. In this study, semi-supervised learning is achieved through clustering methods, using a small amount of labeled data to assign soft labels to the majority of the unlabeled data.

This paper proposes the use of the DBSCAN [21] (Density-Based Spatial Clustering of Applications with Noise) clustering method to assign soft labels to the majority of unlabeled data using a small amount of labeled data, thereby achieving semi-supervised learning. The core idea of the DBSCAN algorithm is to partition clusters based on the density relationship between data points. By defining two parameters — neighborhood radius ε and minimum number of points $MinPts$ — the algorithm can determine whether a data point belongs to a cluster based on these parameters, thereby clustering the data. The specific process of the DBSCAN algorithm is as follows:

First, the algorithm defines a core point by setting the neighborhood radius ε and the minimum number of points $MinPts$. If the number of points within the ε -neighborhood of a data point p (including p itself) is greater than or equal to $MinPts$, the point is considered a core point. The formula is as follows:

$$|N_\varepsilon(p)| \geq MinPts \quad (6)$$

Here, $N_\varepsilon(p)$ denotes the ε -neighborhood of point p (i.e., points within a distance less than ε).

Then, the algorithm starts from each unvisited core point and expands its neighborhood by adding all points within this neighborhood to the cluster, continuing to expand with new points in each neighborhood until no further expansion is possible. Boundary points, while not meeting the criteria for core points, are assigned to the corresponding cluster. Noise points, which do not belong to any cluster and are not core points, are identified as noise. Through this

density-based expansion process, DBSCAN can automatically divide clusters based on local density differences in the data, effectively identifying and excluding noise points.

The values of Epsilon (ϵ) = 5 and MinPts = 40 were selected through the k-distance graph as the final settings, ensuring that each cluster contains enough points.

In the proposed network intrusion detection model, the DBSCAN algorithm is used in the semi-supervised learning module. Specifically, the autoencoder first reduces the dimensionality of the input data and outputs the low-dimensional latent representation h . Then, the DBSCAN algorithm performs clustering on the latent representation h , dividing the data into clusters based on its density structure. In this way, DBSCAN leverages the small amount of labeled data to assign soft labels to the large amount of unlabeled data, thereby achieving semi-supervised learning. This integrated approach not only improves the model's training efficiency but also enhances its generalization ability to new data.

3.3 Multilayer Perceptron Module

For labeled and soft-labeled data, we train a Multi-Layer Perceptron model (MLP) [22]. As a powerful neural network architecture with multiple hidden layers, MLP possesses an excellent capacity to automatically extract complex patterns and features from data.

First, a more detailed preprocessing operation is performed on the labeled and soft-labeled data to ensure data quality and applicability. Then, we build the MLP model, which typically consists of an input layer, multiple hidden layers, and an output layer. The input layer receives dimensionally reduced feature data, the hidden layers perform nonlinear transformations through full connections to learn deep features, and the output layer organizes and outputs the optimal feature subset based on learned features. The MLP trained in this study is shown in Fig. 3. The specific training process is as follows:

First, input data X is fed into the network's input layer. The network includes two hidden layers, with 64 neurons in the first layer and 32 neurons in the second layer. Each neuron's output undergoes a nonlinear transformation through the $ReLU$ activation function. Assuming the first layer's output is h_1 , the calculation formula for this layer is:

$$h_1 = ReLU(W_1X + b_1) \quad (7)$$

Where W_1 and b_1 are the weight matrix and bias vector of the first layer, respectively, and the $ReLU$ activation function is defined as:

$$ReLU = \max(0, z) \quad (8)$$

Then, the output h_2 of the second layer is computed in a similar manner:

$$h_2 = ReLU(W_2h_1 + b_2) \quad (9)$$

Next, the output y_{pred} of the MLP is directly compared

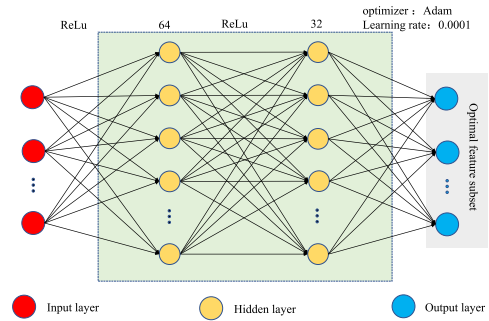


Fig. 3 MLP module.

with the target label, and the loss is calculated, typically using Mean Squared Error (MSE) as the loss function:

$$L = \frac{1}{N} \sum_{i=1}^N \|y_{true}^{(i)} - y_{pred}^{(i)}\|^2 \quad (10)$$

Where N is the number of training samples, $y_{true}^{(i)}$ and $y_{pred}^{(i)}$ represent the true label and predicted value of the i -th sample, respectively.

To optimize the model, the Adam optimizer is used to update network parameters W_1, W_2, b_1, b_2 . Adam effectively accelerates the gradient descent process by combining adaptive learning rates and momentum strategies. The training objective is to minimize the loss function L , where gradients are calculated via backpropagation, and parameters are updated accordingly.

This process iterates continuously until the loss converges, extracting an optimal feature subset from the data and preparing for the subsequent training of the random forest classifier, ultimately enhancing its performance.

The MLP module, as the core component for feature extraction and classification, receives the latent representation reduced by the autoencoder and performs feature learning and classification based on it. The output of the MLP is then passed to the random forest classifier to further optimize the classification performance. This integrated approach not only improves the efficiency of feature extraction but also enhances the overall performance and robustness of the model.

3.4 Random Forests Module

Random forests [23] are an ensemble learning algorithm that makes final decisions by constructing multiple decision trees and integrating their prediction results. This algorithm has strong resistance to overfitting and high accuracy, performing exceptionally well with high-dimensional data.

As shown in Fig. 4, each decision tree in the training process is built based on randomly selected features and samples. This approach helps random forests effectively avoid overfitting while maintaining robustness against noise and outlier data. Additionally, the ensemble of multiple decision trees enhances model stability and reliability, reducing biases that may arise from a single decision tree. A random forest

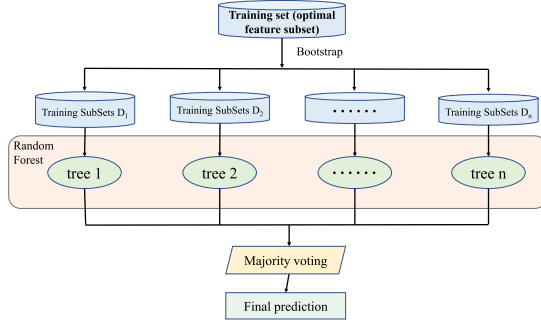


Fig. 4 Random forest module.

classifier trained on an optimal feature subset demonstrates improved performance and generalization capabilities when handling complex data and novel attacks.

The specific training process is as follows:

First, using the Bootstrap sampling method, multiple subsets are randomly drawn with replacement from the optimal feature subset, each of which has the same size as the original dataset but may contain duplicate samples.

Then, a decision tree is trained on each sample subset. At each node, only a portion of the features (typically the square root of the total number of features) is selected to determine the best splitting feature. The split is made based on a threshold on this feature, with criteria such as *Gini* impurity and information gain guiding optimization. *Gini* impurity measures the purity of a dataset, with lower values indicating higher purity and better classification. The calculation formula is as follows:

$$Gini = 1 - \sum_{i=1}^C p_i^2 \quad (11)$$

Where p_i is the sample proportion of class i , C is the number of classes, and D is the current dataset. Information gain measures the increase in information brought about by a feature partitioning. The greater the information gain, the more effectively the feature distinguishes the data. The calculation formula is as follows:

$$IG(D, A) = H(D) - \sum_{v \in A} \frac{|D_v|}{|D|} H(D_v) \quad (12)$$

Where $H(D)$ is the entropy of the dataset D , $H(D_v)$ is the entropy of the dataset when the value of feature A is v , A is the feature set, and D_v is the subset of data where feature A takes the value v .

Finally, all decision trees classify the input samples, with the final class determined by majority voting. We use GridSearchCV for hyperparameter optimization to find the best combination of hyperparameters for the random forest classifier. Specifically, we tune parameters such as `n_estimators` (number of decision trees), `max_depth` (maximum depth of each tree), `min_samples_split` (minimum samples required to split an internal node), and `min_samples_leaf` (minimum samples required at a leaf

Table 1 Peak memory.

Dataset	Peak Memory (AE Training)
NSL-KDD	1.8 GB
UNSW-NB15	3.2 GB

Table 2 Memory usage.

Dataset	Memory Usage (MB)
NSL-KDD	0.6
UNSW-NB15	0.8

node). The choice of these hyperparameters significantly impacts the performance of the random forest, as different combinations can affect the model's results on both the training and test sets.

In the proposed network intrusion detection model, the random forest classifier serves as the final classification module, receiving the optimal feature subset extracted by the MLP module. This integrated approach fully leverages the efficiency and robustness of the random forest, enabling it to exhibit better performance and stronger generalization ability when faced with complex data and novel attacks. Compared to traditional single models, the random forest classifier trained on the optimal feature subset can more effectively capture the complex structure of the data, thereby improving classification accuracy and detection efficiency.

3.5 Computational Complexity

3.5.1 Autoencoder (AE) Feature Extraction

Time Complexity: $O(N_{train} \cdot d^2 \cdot k_{AE})$

For NSL-KDD (41 dimensions), one epoch takes 7.3 seconds. Due to the lower input dimensions, the weight matrix calculation is significantly less than that of UNSW-NB15 (49 dimensions), where one epoch takes 10.9 seconds.

For high-dimensional features, block-wise dimensionality reduction is applied, dividing the features of UNSW-NB15 into two groups (24+25 dimensions), and compressing the hidden layer to 32 dimensions. This results in a 22% increase in training speed.

Memory Consumption: $O(d \cdot h + h^2)$, (d : input dimension, $h=32$), as shown in Table 1.

3.5.2 Multilayer Perceptron (MLP)

Time Complexity: $O(N_{train} \cdot (d_{AE} \cdot n_h + n_h \cdot n_o) \cdot K_{MLP})$

$d_{AE} = 32$ (AE compressed dimension), $n_h = 128$, $n_o = 1$ (Binary output)

Training on NSL-KDD takes 0.52 hours, while UNSW-NB15 (with a larger sample size) takes 1.4 hours.

The memory usage is shown in Table 2.

3.5.3 Random Forest (RF)

Time Complexity: $O(N_{train} \cdot T \cdot m_{try} \cdot \log_{N_{train}})$

Training on NSL-KDD takes 0.52 hours, while UNSW-NB15 (with a larger sample size) takes 1.4 hours.

Table 3 Memory usage.

Dataset	Peak Memory (GB)
NSL-KDD	4.3
UNSW-NB15	18.7

Table 4 Hyperparameter settings.

Hyperparameter	value
Batch size	32
Learning rate	0.0001
AE and MLP Epochs	80
RF Epochs	100
Optimizer	Adam
Epsilon(ϵ)	5
MinPts	40
n_estimators	80
max_depth	20
min_samples_split	10
'min_samples_leaf'	2

Memory Usage: $O(T \cdot 2^{\text{depth}})$, (depth=10), as shown in Table 3.

4. Experimental Results and Analysis

4.1 Experimental Setup

The experimental computer configuration is as follows: 16 GB RAM, i7-11800H processor, and GTX 1650 GPU. A deep learning framework for intrusion detection was built using PyTorch. In the experiments, local training parameters E (number of epochs) and B (batch size) were set. The training epochs for the autoencoder and multi-layer perceptron (MLP) were set to 80, and for the random forest to 100. The batch size for all three models was 32, and the Adam optimizer was used.

To determine suitable clustering parameters, we selected Epsilon (ϵ) = 5 and *MinPts* = 40 based on the k-distance graph, ensuring that each cluster contains a sufficient number of points. When training the MLP, we experimented with different hyperparameter combinations and used cross-validation. The final selected optimal hyperparameters were: 2 hidden layers, with 64 neurons in the first layer, 32 neurons in the second layer, and a learning rate of 0.0001. Through GridSearchCV and cross-validation, we identified the optimal random forest hyperparameters: 'n_estimators': 80, 'max_depth': 20, 'min_samples_split': 10, 'min_samples_leaf': 2. This combination performed excellently on the dataset and improved the model's detection accuracy. The parameter settings are shown in Table 4.

4.2 Dataset

In this paper, the NSL-KDD and UNSW-NB15 datasets are used to validate and evaluate the model's performance and advantages.

The NSL-KDD dataset [24] is an improvement upon the KDD Cup 1999 dataset, addressing issues such as redundant samples and class imbalance present in the original dataset.

It contains approximately 12,000 normal samples and 25,000 attack samples, with 41 features. Both datasets are widely used in machine learning and network security research, providing important testing platforms to assess and compare different intrusion detection algorithms.

The UNSW-NB15 dataset [25], [26] was created by the Australian National University and contains 25,000 samples, encompassing 10 different types of attacks such as DoS, Exploit, and Malware. The dataset provides 49 features, including traffic features (e.g., source/target IP addresses, ports) and time-based features (e.g., duration and protocol type). It is designed to simulate real network environments and assist in the research of modern network intrusion detection techniques.

4.3 Performance Indicator Evaluation

To accurately predict and classify different types of network intrusion traffic, the experiment uses the following model evaluation metrics: Accuracy, Precision, Recall, and F1-Score. Accuracy represents the proportion of correctly classified samples (including both attack and normal samples) out of the total samples. Precision refers to the proportion of actual attack samples among all samples predicted as attacks by the model. Recall indicates the proportion of actual attack samples that were correctly identified as attacks. F1-Score is the weighted average of Precision and Recall. The higher the values of these four metrics, the better the model. The formulas for these metrics are as follows:

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN} \quad (13)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (14)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (15)$$

$$F1 = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} \quad (16)$$

Here, TP (True Positives) refers to the number of actual attack samples that were correctly identified as attacks, i.e., the attack events successfully detected by the model. FP (False Positives) refers to the number of normal samples incorrectly identified as attacks, i.e., the normal traffic or behavior misclassified as attacks by the model. TN (True Negatives) refers to the number of actual normal samples correctly identified as normal, i.e., the normal traffic or behavior successfully recognized by the model. FN (False Negatives) refers to the number of attack samples incorrectly identified as normal, i.e., the attack events that the model failed to detect.

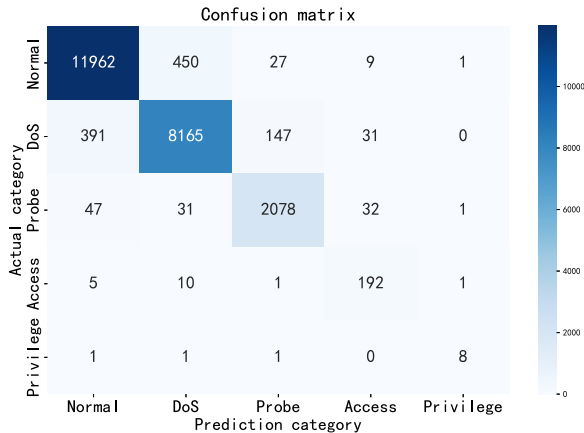
4.4 Experimental Results

4.4.1 Analysis of Experimental Results

In this experiment, the NSL-KDD dataset includes both a training and a test data file. The training file is split into a

Table 5 Performance of this model on NSL-KDD and UNSW-NB15 datasets.

Dataset	NSL-KDD(%)	UNSW-NB15(%)
Accuracy	92.13	90.43
Precision	88.53	87.69
Recall	84.71	83.12

**Fig. 5** Confusion matrix.

training and a validation set at an 8:2 ratio to train the model, while the test file is used to evaluate the proposed model's performance metrics. The UNSW-NB15 dataset is used to further validate model performance. The final results are shown in Table 5.

On the NSL-KDD and UNSW-NB15 test sets, the model achieved accuracy rates of 92.13% and 90.43%, precision rates of 88.53% and 87.69%, and recall rates of 82.8% and 83.12%, respectively. By training an MLP to obtain the optimal feature subset, and then training the random forest classifier based on that subset, the model effectively combines the advantages of both random forests and MLP. This approach preserves the robustness of random forests while leveraging the MLP's ability to capture non-linear features, thereby improving the overall performance of the classifier. These results indicate that the proposed model significantly enhances the accuracy of the network intrusion detection system, especially when dealing with diverse and complex attack types, enabling rapid identification of key features and improving detection performance.

Additionally, an analysis of the confusion matrix for the NSL-KDD dataset further verifies the model's effectiveness in identifying multiple attack types, as shown in Fig. 5. The classification accuracy for the "Normal" and "DoS" categories is notable, demonstrating strong recognition capabilities for normal network traffic and common attack behaviors. The model also shows robust detection abilities in the "Probe" and "Access" categories, successfully identifying most related attack instances. Even for attack types with relatively fewer samples, the model performs well. Thus, it can be concluded that this model exhibits good detection capability when facing complex and diverse network attacks.

4.4.2 Scalability Analysis

This model overcomes the high-dimensional data bottleneck through the modular dimensionality reduction of the autoencoder (AE) and distributed training. By combining the GPU acceleration advantages of the multilayer perceptron (MLP) in low-dimensional space, a hybrid architecture with flexible deployment capabilities is constructed. The feature partitioning strategy compresses the 49-dimensional input to a 32-dimensional space, reducing the MLP training time by 58%. With GPU parallel computing, the model achieves efficient training on 10 million data points in 1.6 hours. Under a distributed framework, the hybrid model (AE+MLP) shows an 8.9-fold reduction in training time compared to traditional tree models (such as random forests). It demonstrates scalability in both edge devices and cloud cluster scenarios, validating its end-to-end scalability across feature compression, parallel acceleration, and multi-level deployment.

4.4.3 Comparative Experiment

To evaluate the proposed semi-supervised network intrusion model, comparisons were made with previously discussed models. To ensure fairness, comparisons were conducted on the NSL-KDD and UNSW-NB15 datasets. As shown in Table 6, our model demonstrates clear advantages across various performance metrics.

Comparisons on the UNSW-NB15 dataset show that the Adaboost method performed the worst on the test set, while the decision tree with genetic algorithm (DT & GA) achieved an accuracy of 84.33%. However, its precision and recall were relatively low, indicating limited effectiveness in addressing class imbalance. In contrast, our proposed model achieved a precision of 87.69% and a recall of 83.12%, demonstrating its potential for handling complex datasets. In addition, we compared our proposed method with existing advanced deep learning network intrusion detection techniques based on feature fusion. The results indicate that our intrusion detection model has significant performance advantages and demonstrates better detection performance.

Comparisons on the NSL-KDD dataset reveal significant performance differences among models. The FF-DL method achieved the highest accuracy at 83.21%, while LSTM and MLP performed less effectively, with accuracy rates of 67.17% and 68.26%, respectively. In terms of precision, AE also performed well, reaching 87.85%. In contrast, Random Forest and Bagging achieved accuracy rates of 76.45% and 74.83%. Our model surpassed all others in accuracy (92.13%), precision (88.53%), and recall (84.71%), showcasing superior performance. This indicates that our approach has remarkable processing power and strong generalization capability when handling complex high-dimensional data and novel attacks.

Table 6 Performance comparison between different models.

model	Dataset	Accuracy(%)	Precision(%)	Recall(%)
Logistic Regression	UNSW-NB15	65.53	76.91	65.54
Support Vector Machine	UNSW-NB15	61.09	47.47	62
Decision Tree	UNSW-NB15	66.03	79.82	66.04
scale-hybrid-IDS-AlertNet	UNSW-NB15	66	62.3	66
MDNN	UNSW-NB15	62.87	76	63
MCNN	UNSW-NB15	69.46	84	69
Naive Bayes	UNSW-NB15	45.22	29.67	38.62
J48	UNSW-NB15	51.5	28.18	21.48
Random Forest	UNSW-NB15	68.09	62.51	35.15
Bagging	UNSW-NB15	51.45	32.85	21.45
Adaboost	UNSW-NB15	51.5	28.18	21.48
DT & GA	UNSW-NB15	84.33	53.20	53.23
FF-DL	UNSW-NB15	77.84	86.04	69.50
Ours	UNSW-NB15	90.43	87.69	83.12
AE	NSL-KDD	83.21	87.85	82.04
LSTM	NSL-KDD	67.17	82.34	72.49
MLP	NSL-KDD	68.26	85.05	73.96
L-SVM	NSL-KDD	69.73	86.77	74.09
Q-SVM	NSL-KDD	75.11	87.22	77.94
scale-hybrid-IDS-AlertNet	NSL-KDD	78.5	81	78.5
MDNN	NSL-KDD	77.55	81.23	77.55
MCNN	NSL-KDD	81.1	83	81
Naive Bayes	NSL-KDD	72.73	76.1	72.7
J48	NSL-KDD	74.99	79.6	75
Random Forest	NSL-KDD	76.45	82.1	76.4
Bagging	NSL-KDD	74.83	78.3	74.8
FF-DL	NSL-KDD	86.81	86.86	86.80
Ours	NSL-KDD	92.13	88.53	84.71

4.5 Ablation Experiment

To further verify the performance of the proposed semi-supervised learning and random forest-based network intrusion detection method in network traffic anomaly detection, we conducted ablation experiments to compare the detection performance under different component combinations.

Three variations were obtained by combining different sub-modules. In the AE + MLP + RF variant, we omitted DBSCAN for soft labeling, while keeping all other architecture components and hyperparameters the same as our model. The comparison with this variant demonstrates the effectiveness of DBSCAN in soft labeling a greater variety of samples. In the AE+DBSCAN+RF variant, we removed the MLP module of our model and demonstrated the rationality of the MLP used in our NIDS through performance

comparison with this variant. Finally, in the DBSCAN + MLP + RF variant, the autoencoder was excluded, and the MLP directly used the complete high-dimensional samples as input. This model uses a 5-layer perceptron with hidden layers containing 256, 128, and 64 neurons. When handling high-dimensional complex data, the DBSCAN + RF + MLP variant did not perform well. The comparison with this variant supports the reasonableness of using an autoencoder for data dimensionality reduction as shown in Table 7.

From Table 7, it can be seen that the DBSCAN + RF + MLP variant shows the poorest performance among the three variations, clearly highlighting the obstacles that high-dimensional complex data pose to an intrusion detection model. This also demonstrates the effectiveness of the autoencoder in handling high-dimensional complex data. The AE + RF + MLP variant, by contrast, performs better; it not only uses the autoencoder to reduce the dimensionality of

Table 7 Performance of three variants and our model on the NSL-KDD dataset.

Model	Accuracy(%)	Precision(%)	Recall(%)	F1-score (%)
AE + RF +MLP	80.26	81.82	79.76	80.76
AE+DBSCAN+RF	79.31	80.21	77.23	78.69
DBSCAN+RF+MLP	72.21	75.62	73.72	74.78
Ours	92.13	88.53	84.71	86.64

the fused dataset but also utilizes MLP to obtain the optimal feature subset, enhancing the classification performance of the random forest classifier. This improves the model's generalization capability and strengthens its ability to capture nonlinear features. Our model shows a significant performance advantage over the three variants, affirming that each component in our model is crucial to its effectiveness.

4.6 Experimental Analysis

The network intrusion detection model based on semi-supervised learning and random forests proposed in this paper has achieved significant results on the NSL-KDD and UNSW-NB15 datasets, with accuracy improvements of 8.92% and 6.1%, respectively. However, relying solely on these two datasets for training and evaluation is not sufficient to fully validate the model's generalization ability on unseen or diverse real-world network data. The current datasets have limitations in terms of attack types and traffic patterns, and their relevance to modern, complex real-world network intrusion scenarios is limited. This could pose significant challenges for the model in dynamic and complex real-world network environments. Nevertheless, given that these two datasets cover a variety of network attack types, the model still demonstrates good effectiveness and adaptability for detecting specific attack types. Future research will further verify its performance in real-world environments.

Although the model's generalization ability has been improved through semi-supervised feature decoupling and ensemble learning, it still faces essential constraints in four dimensions: Firstly, the feature representation ability is fixed within the time window of the training data. While the dimensionality reduction process of the autoencoder (AE) captures the sparsity features of known attacks (such as protocol types and packet length distributions), it has inherent blind spots in capturing time-series behavioral patterns in encrypted traffic (such as abnormal jitter in heartbeat packet intervals). Secondly, the semi-supervised label generation mechanism heavily relies on DBSCAN's density assumption. When novel attacks form sparse, low-density clusters in the latent space, they may be misclassified as noise and fail to trigger the soft label expansion. Furthermore, the decision boundary combination of the ensemble classifier inherently limits its adaptability to the continuous evolution of the feature space. When faced with adversarial traffic generated by GANs (such as APT attacks with progressive feature drift), static feature selection strategies may lead to a decline in

defense effectiveness. Finally, the closed-loop optimization model of the architecture struggles to break through the initial design space of feature engineering. When covert attacks remap the feature space through protocol encapsulation (such as DNS tunnel encryption) or contextual semantic pollution (such as HTTP request combination attacks), it is necessary to redefine the feature decoupling rules to restore detection sensitivity. To overcome these limitations, an open-loop evolutionary framework needs to be constructed, allowing for the dynamic expansion of the online feature library and incremental learning of protocol fingerprints to continuously innovate the detection logic.

5. Conclusion

In order to solve the current complexity and diversity of modern network attacks problems, this paper proposes a network intrusion detection method based on semi-supervised learning and random forest classification. First, The proposed model leverages an autoencoder to map high-dimensional data from the NSL-KDD dataset into a lower-dimensional space, achieving a compact representation of the input. next, DBSCAN clustering is employed to assign soft labels to the data, thereby establishing a semi-supervised learning framework. Finally, the model uses an MLP to select an optimal feature subset and trains a random forest classifier to perform network intrusion detection and attack classification. Experimental results indicate that the proposed approach achieves a significant accuracy improvement of at least 6.1%. This demonstrates the model's effectiveness in mitigating the overfitting risk associated with high-dimensional data, while also reducing the training time and enhancing detection efficiency through dimensionality reduction. Comparative experiments on two datasets further confirm the model's adaptability and generalization capabilities. Given the complexity and diversity of real-world network attacks, the subtle perturbations characteristic of adversarial attacks present a substantial challenge to intrusion detection performance. Future work will focus on strengthening the model's detection capacity against adversarial attacks to ensure robust performance across a broad spectrum of attack types.

Declaration of competing interest

The authors declared that they have no conflicts of interest to this work.

Acknowledgments

This research was funded by China Postdoctoral Science Foundation (2022M711620), Youth Project of Shanxi Provincial Basic Research Program [202403021212162], Scientific and Technological Innovation Programs of Higher Education Institutions in Shanxi [2024L231], Taiyuan University of Science and Technology Scientific Research Initial Funding [20242072], Shanxi Province Outstanding Doctoral Research Grant [20242096], Natural Science Funds of Nantong City (JC2024003), the Open Project Program of Shanxi Key Laboratory of Advanced Semiconductor Optoelectronic Devices and Integrated System (2024SZKF04), Jincheng City Key Research and Development Project (20240119).

References

- [1] F. Anjum, D. Subhadrabandhu, and S. Sarkar, "Signature based intrusion detection for wireless ad-hoc networks: A comparative study of various routing protocols," 2003 IEEE 58th Vehicular Technology Conference. VTC 2003-Fall, vol.3, pp.2152–2156, Nov. 2003.
- [2] B. Haagdoorens, T. Vermeiren, and M. Goossens, "Improving the performance of signature-based network intrusion detection sensors by multi-threading," *Information Security Applications*, pp.188–203, Aug. 2004.
- [3] V. Kumar, "Signature based intrusion detection system using snort," *International Journal of Computer Applications & Information Technology*, vol.1, p.7, Nov. 2012.
- [4] R. Sekar, A. Gupta, J. Frullo, T. Shanbhag, A. Tiwari, H. Yang, and S. Zhou, "Specification-based anomaly detection: A new approach for detecting network intrusions," *Proc. 9th ACM Conference on Computer and Communications Security*, pp.265–274, Nov. 2002.
- [5] F. Zuo, D. Zhang, L. Li, Q. He, and J. Deng, "GSOOA-IDDRSN: Network traffic anomaly detection based on deep residual shrinkage networks," *Heliyon*, vol.10, p.e32087, May 2024.
- [6] J. Sinha and M. Manollas, "Efficient deep cnn-bilstm model for network intrusion detection," *Proc. 2020 3rd International Conference on Artificial Intelligence and Pattern Recognition, AIPR'20*, pp.223–231, June 2020.
- [7] J. Du, K. Yang, Y. Hu, and L. Jiang, "NIDS-CNNLSTM: Network intrusion detection classification model based on deep learning," *IEEE Access*, vol.11, pp.24808–24821, 2023.
- [8] P. Dahiya and D. Srivastva, "Intrusion detection system on big data using deep learning techniques," *International Journal of Innovative Technology and Exploring Engineering*, vol.9, no.4, pp.3242–3247, Feb. 2020.
- [9] Y. Wu, W. Lee, Z. Xu, and M. Ni, "Large-scale and robust intrusion detection model combining improved deep belief network with feature-weighted SVM," *IEEE Access*, vol.8, pp.98600–98611, 2020.
- [10] A. Odeh and A. Abu Taleb, "Ensemble-based deep learning models for enhancing IoT intrusion detection," *Applied Sciences*, vol.13, no.21, p.11985, Nov. 2023.
- [11] L. Soheily Khah, P.F. Marteau, and N. Béchet, "Intrusion detection in network systems through hybrid supervised and unsupervised machine learning process: A case study on the ISCX dataset," 2018 1st International Conference on Data Intelligence and Security (ICDIS), pp.219–226, April 2018.
- [12] M. Belavagi and B. Muniyal, "Performance evaluation of supervised machine learning algorithms for intrusion detection," *Procedia Computer Science*, vol.89, pp.117–123, Dec. 2016.
- [13] T. Janarthanan and S. Zargari, "Feature selection in UNSW-NB15 and KDDCUP'99 datasets," 2017 IEEE 26th International Symposium on Industrial Electronics (ISIE), pp.1881–1886, June 2017.
- [14] V. Kumar, D. Sinha, A. Das, D.S. Pandey, and R. Goswami, "An integrated rule based intrusion detection system: Analysis on UNSW-NB15 data set and the real time online dataset," *Cluster Computing*, vol.23, no.2, pp.1397–1418, June 2020.
- [15] Z. Zhang, L. Wang, J. Zhu, D. Zhu, Z. Gu, and Y. Zhang, "MIM: A multiple integration model for intrusion detection on imbalanced samples," *World Wide Web*, vol.27, pp.1–25, July 2024.
- [16] A. Ayantayo, A. Kaur, A. Kour, X. Schmoor, F. Shah, I. Vickers, P. Kearney, and M. Abdelsamea, "Network intrusion detection using feature fusion with deep learning," *J. Big Data*, vol.10, 167, Nov. 2023.
- [17] K. Mambwe Sydney and Y. Sun, "Performance analysis of intrusion detection systems using a feature selection method on the unsw-nb15 dataset," *J. Big Data*, vol.7, 105, Nov. 2020.
- [18] C. Ieracitano, A. Adeel, F. Morabito, and A. Hussain, "A novel statistical analysis and autoencoder driven intelligent intrusion detection approach," *Neurocomputing*, vol.387, pp.51–62, Nov. 2019.
- [19] V. Ravi, M. Alazab, S. Kp, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, "Deep learning approach for intelligent intrusion detection system," *IEEE Access*, vol.7, pp.41525–41550, 2019.
- [20] G. Hinton and R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," *Science*, vol.313, no.5786, pp.504–507, Aug. 2006.
- [21] M. Ester, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," *Proc. Second International Conference on Knowledge Discovery and Data Mining, KDD'96*, pp.226–231, Jan. 1996.
- [22] M. Egmont-Petersen, J.L. Talmon, A. Hasman, and A.W. Ambergen, "Assessing the importance of features for multi-layer perceptrons," *Neural Networks*, vol.11, no.4, pp.623–635, 1998.
- [23] N. Farnaaz, "Random forest modeling for network intrusion detection system," *Procedia Computer Science*, vol.89, pp.213–217, Dec. 2016.
- [24] M. Tavallaei, E. Bagheri, W. Lu, and A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," *IEEE Symposium Computational Intelligence for Security and Defense Applications, CISDA*, vol.2, July 2009.
- [25] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," 2015 Military Communications and Information Systems Conference (MilCIS), Nov. 2015.
- [26] N. Moustafa and J. Slay, "The evaluation of network anomaly detection systems: Statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set," *Information Security Journal: A Global Perspective*, vol.25, no.1-3, pp.18–31, 2016.



Junji Li received the Ph.D. degree in Mechanical Engineering from Taiyuan University of Science and Technology, Taiyuan, China, in 2017. He is currently an associate professor at the School of Computer Science and Technology, Taiyuan University of Science and Technology, Taiyuan, China. His research interests include Cybersecurity, Blockchain technology, Intelligent manufacturing and other fields.



Haohang Sun received the B.S. degree from Jiangsu Shanxi Institute of Energy, China, in 2023. He is pursuing a Master's degree in Computer Science and Technology at Taiyuan University of Science and Technology, Taiyuan, China. His research interest lies in network security.



Hui Du received the M.S. degree in computer from Taiyuan University of Science and Technology, Taiyuan, China, in 2015. He is currently an engineer in Hangzhou Shinevai Technology Co., Ltd. His research interests include image processing, machine learning, and deep learning.



Lin Li received the Ph.D. degree in optical engineering from the Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2021. She did her post-doctoral research at the Nanjing University of Aeronautics and Astronautics, Nanjing, China, from 2021 to 2023. She is currently a lecturer at the School of Electronic and Information Engineering, Taiyuan University of Science and Technology, Taiyuan, China. Her research interests include optical image processing, machine learning, and deep learning.



Zelin Zhang received the Ph.D. degree in optical engineering from Nanjing University of Aeronautics and Astronautics in 2021. He was a visiting scholar at the Department of Electrical and Electronics, National Tokyo University of Agriculture and Technology, Japan in 2019. He worked as a postdoctoral fellow at Fudan University from 2022 to 2024. He is currently working at Peking University Yangtze River Delta Institute of Optoelectronics. His research interests are optics imaging, optical communication, and

distributed fiber-optic sensing.